

Backend Schema Document

MongoDB collections, fields, indexes, relationships and data rules

Complete data model for the Gymholik backend: conventions, ER diagram, every collection with fields and indexes, enums, denormalisation rules, query-to-index map, aggregation examples, PII classification, Java entity skeletons and sample documents.

Version 1.0 · 8 October 2026

Prepared for: Product owner, Gymholik

Status: Working draft for build and pilot

Confidential. Do not share without permission.

Contents

- 1. Data Model Overview 3
- 2. Enumerations 4
- 3. Collections in Detail 5
- 4. Consistency and Denormalisation Rules 11
- 5. Query to Index Map 11
- 6. Aggregation and Query Examples 12
- 7. PII Classification and Masking 12
- 8. Validation Constraints 13
- 9. Java Entity Skeletons 13
- 10. Sample Documents 14
- 11. Database Setup and Migration 14

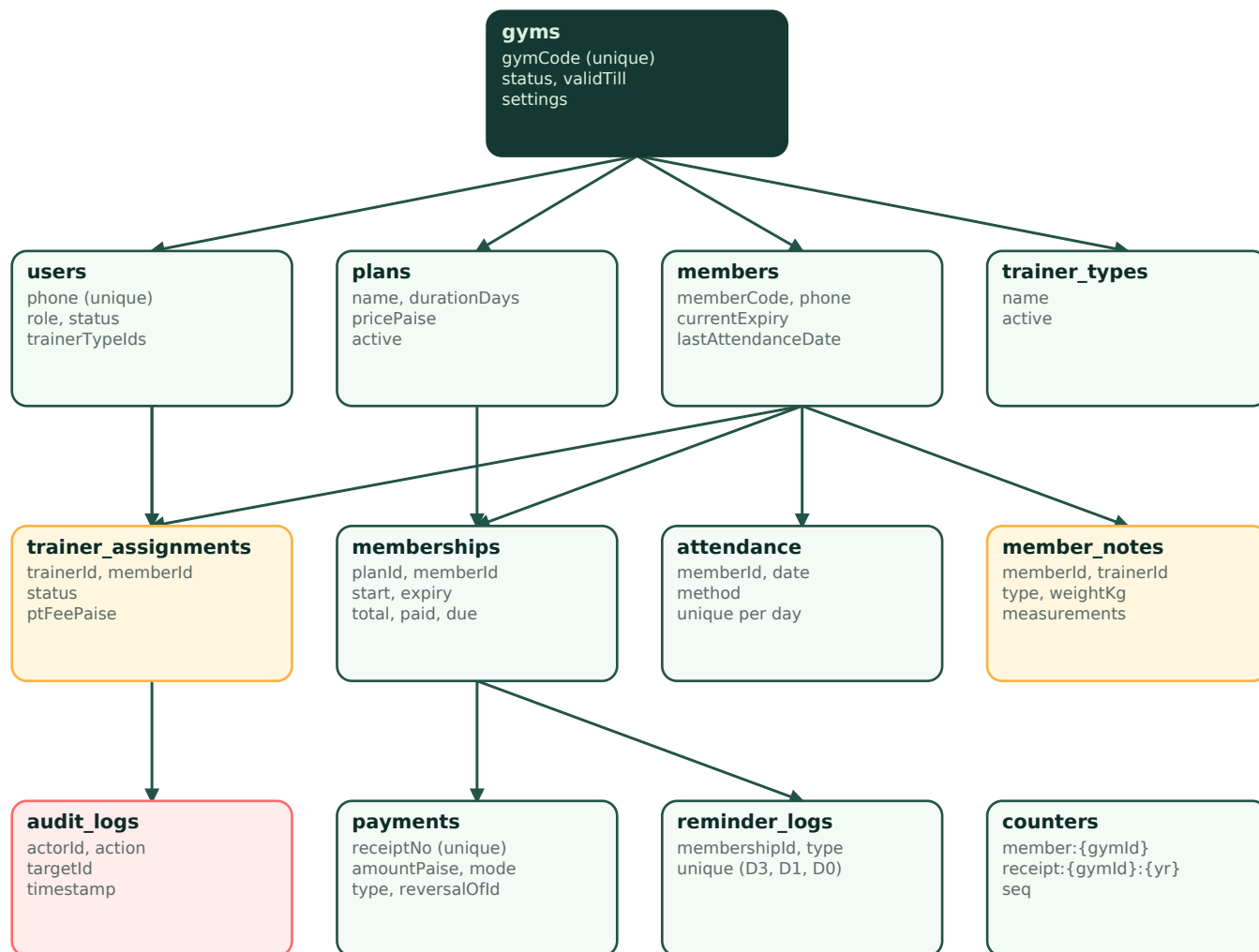
1. Data Model Overview

Database: **gymdb** on MongoDB 7 (single-node replica set so multi-document transactions work). One database for all gyms (shared collections). Every tenant-owned document has a **gymId**. Two collections are global: gyms (the tenant itself) and counters.

1.1 Conventions

Topic	Rule
Collection names	snake_case plural: gyms, users, plans, members, memberships, payments, attendance, reminder_logs, counters, trainer_types, trainer_assignments, member_notes, audit_logs
Field names	camelCase
Primary key	_id ObjectId (String in Java). Counters use a string key
Tenant key	gymId (String ObjectId) in every tenant document, from TenantDocument base class. Users of role SUPER_ADMIN have gymId null
Money	long in paise, field names end with Paise. Reversals are negative amounts. Never double, never BigDecimal strings
Business dates	LocalDate stored as ISO date (yyyy-MM-dd) interpreted in Asia/Kolkata: startDate, expiryDate, joiningDate, dob, attendance date, subscriptionValidTill
Event times	Instant (UTC): createdAt, updatedAt, paidAt, checkInTime, sentAt, timestamp
Audit fields	createdAt and updatedAt via Spring Data auditing (@EnableMongoAuditing)
Deletes	No hard delete for members, payments, memberships, attendance. Use status fields. Notes and logs are append-only
Enums	Stored as strings in UPPER_SNAKE_CASE
Searchable names	Keep a lower-case copy (nameLower) for prefix search with an index, and escape user input
Schema version	Optional schemaVersion int on documents that may change; migrations run at startup (idempotent)

1.2 Entity-relationship diagram



Every collection except counters also carries gymId (omitted from the drawing for clarity). users.trainerTypeIds references trainer_types. member_notes.trainerId and trainer_assignments.trainerId reference users. payments.memberId references members. Amber boxes are trainer-related. Coral is audit.

2. Enumerations

Enum	Values	Notes
Role	SUPER_ADMIN, OWNER, STAFF, TRAINER	TRAINER is separate from STAFF
UserStatus	ACTIVE, PENDING_APPROVAL, REJECTED, INACTIVE	Replaces the old boolean "active". Only ACTIVE can log in
GymStatus	TRIAL, ACTIVE, EXPIRED, SUSPENDED	Derived from validTill by a daily job and on read
SubscriptionPlan	TRIAL, MONTHLY, YEARLY	
MemberStatus	ACTIVE, INACTIVE	INACTIVE = soft deleted
MembershipStatus	ACTIVE, EXPIRED	Must exist. Dashboard and reminder queries depend on it

Enum	Values	Notes
PaymentMode	CASH, UPI, CARD	
PaymentType	PAYMENT, REVERSAL	
CheckInMethod	MANUAL, QR	
AssignmentStatus	ACTIVE, ENDED	
NoteType	WORKOUT, DIET, PROGRESS, GENERAL	
ReminderType	D3, D1, D0	Days before expiry (0 = on the day)

3. Collections in Detail

Legend: R = required. U = unique. Types are Java types. "Idx" lists indexes for that collection.

3.1 gyms

The tenant. Not a TenantDocument (it is the tenant). Holds subscription state and gym-level settings.

Field	Type	R	Notes
_id	ObjectId	R	Becomes gymId everywhere else
name	String	R	Display name
ownerName	String	R	
phone	String	R	Gym contact. Same as owner mobile (no separate gym phone in the UI)
address	String		Optional, set later in Gym settings
logoPath	String		File path under uploads/{gymId}
gymCode	String	R U	6 to 8 chars, no look-alike characters. Used by trainers to join
status	GymStatus	R	TRIAL at creation
subscriptionPlan	SubscriptionPlan	R	TRIAL initially
subscriptionValidTill	LocalDate	R	Trial: today + 14. Extended by SUPER_ADMIN
graceDays	int		Default 5
allowExpiredCheckin	boolean	R	Default false
reminderDaysBefore	int	R	Default 3
staffCanSeeFullPhone	boolean	R	Default false. Never affects TRAINER
timezone	String		Default Asia/Kolkata
createdAt, updatedAt	Instant	R	Auditing

Index	Purpose
gymCode (unique)	Gym code lookup and uniqueness
status + subscriptionValidTill	Daily job and admin list

3.2 users

Everyone who logs in: owner, staff, trainers, super admin. Login ID is the mobile number, globally unique.

Field	Type	R	Notes
_id	ObjectId	R	
gymId	String		Null for SUPER_ADMIN only
name	String	R	

Field	Type	R	Notes
phone	String	R U	10 digits, globally unique (login)
passwordHash	String	R	BCrypt. Never returned by any API
role	Role	R	
status	UserStatus	R	Trainers start PENDING_APPROVAL
trainerTypeIds	List of String		Trainers only. At least one when ACTIVE
approvedBy, approvedAt	String, Instant		Owner approval trail
failedAttempts	int	R	Reset on success
lockedUntil	Instant		15 minutes after 5 failures
refreshTokenHash	String		Hash of the current refresh token (rotation)
tokenVersion	int		Increment to invalidate all tokens
mustChangePassword	boolean		True after owner reset
lastLoginAt	Instant		
createdAt, updatedAt	Instant	R	

Index	Purpose
phone (unique)	Login lookup
gymId + role + status	List staff or trainers, pending requests

3.3 plans

Membership plans defined by each gym.

Field	Type	R	Notes
_id, gymId		R	
name	String	R	Monthly, Quarterly...
durationDays	int	R	30, 90, 180, 365
pricePaise	long	R	
active	boolean	R	False hides it from pickers; old memberships unaffected
createdAt, updatedAt	Instant	R	

Index	Purpose
gymId + active	Plan picker
gymId + name (unique)	Avoid duplicate names

3.4 members

A gym customer. Holds identity data and a few denormalised fields for fast lists.

Field	Type	R	Notes
_id, gymId		R	
memberCode	String	R U	Per gym, like GYM-0045. Content of the QR code
name	String	R	PII, visible to all roles
nameLower	String	R	For prefix search
phone	String	R U	PII. Unique per gym. Masked for STAFF and TRAINER
gender	String		MALE, FEMALE, OTHER

Field	Type	R	Notes
dob	LocalDate		PII. Owner only
address	String		PII. Owner only
photoPath	String		uploads/{gymId}/{uuid}.webp
joiningDate	LocalDate	R	
status	MemberStatus	R	ACTIVE or INACTIVE
currentMembershipId	String		Latest membership
currentExpiry	LocalDate		Denormalised from membership
currentDuePaise	long	R	Denormalised, sum of due across memberships
lastAttendanceDate	LocalDate		Denormalised, drives absent list
createdBy	String		User id
createdAt, updatedAt	Instant	R	

Index	Purpose
gymId + phone (unique)	Duplicate check, search by phone
gymId + memberCode (unique)	QR and code lookup
gymId + status + currentExpiry	Active, expired, expiring-soon lists
gymId + status + lastAttendanceDate	Absent members
gymId + nameLower	Prefix name search
gymId + currentDuePaise (partial, due greater than 0)	Dues list

3.5 memberships

One paid period on a plan. Each renewal creates a new record. Source of truth for dues.

Field	Type	R	Notes
_id, gymId		R	
memberId	String	R	
planId, planName	String	R	Plan name copied so history survives plan edits
startDate, expiryDate	LocalDate	R	expiry = start + durationDays
totalPaise	long	R	Plan price at the time
paidPaise	long	R	Sum of payments minus reversals
duePaise	long	R	total - paid, never negative
status	MembershipStatus	R	ACTIVE or EXPIRED
createdAt, updatedAt	Instant	R	

Index	Purpose
gymId + memberId + startDate desc	Latest membership and history
gymId + status + expiryDate	Daily expiry job, dashboard
gymId + duePaise (partial, due greater than 0)	Dues queries

3.6 payments

Money movements. Append-only. A reversal is a new negative record.

Field	Type	R	Notes
_id, gymId		R	
memberId, membershipId	String	R	
memberName	String	R	Copy for history lists
amountPaise	long	R	Positive for PAYMENT, negative for REVERSAL
mode	PaymentMode	R	
type	PaymentType	R	Default PAYMENT
reversalOfId	String		Original payment id
reason	String		Required for REVERSAL
receiptNo	String	R U	GYMCODE-2026-000123, per gym
idempotencyKey	String		Client UUID to stop duplicates
paidAt	Instant	R	
receivedBy	String	R	User id
note	String		
createdAt	Instant	R	

Index	Purpose
gymId + paidAt desc	History, today and month totals
gymId + memberId + paidAt desc	Member payment history
gymId + receiptNo (unique)	Receipt lookup
gymId + idempotencyKey (unique, partial)	Duplicate prevention

3.7 attendance

One check-in per member per day.

Field	Type	R	Notes
_id, gymId		R	
memberId	String	R	
memberName	String	R	Copy
date	LocalDate	R	Gym local date
checkInTime	Instant	R	
method	CheckInMethod	R	
markedBy	String	R	User id
expiredWarning	boolean	R	True if membership was expired

Index	Purpose
gymId + memberId + date (unique)	One per day
gymId + date desc	Today list and counts

3.8 reminder_logs

Prevents sending the same reminder twice.

Field	Type	R	Notes
_id, gymId		R	
memberId, membershipId	String	R	
type	ReminderType	R	D3, D1, D0
channel	String	R	LOG now; WHATSAPP, SMS later
sentAt	Instant	R	
status	String	R	SENT, FAILED

Index	Purpose
gymId + membershipId + type (unique)	Idempotent daily job

3.9 trainer_types

Owner-defined categories for trainers.

Field	Type	R	Notes
_id, gymId		R	
name	String	R	Personal Trainer, Cardio Trainer, Zumba Trainer, Yoga Trainer, CrossFit Trainer, General
active	boolean	R	
isDefault	boolean		Seeded at gym creation

Index	Purpose
gymId + name (unique)	Avoid duplicates

3.10 trainer_assignments

Which members a trainer may see. Only the owner creates or ends these.

Field	Type	R	Notes
_id, gymId		R	
trainerId	String	R	User of role TRAINER, ACTIVE
memberId	String	R	
status	AssignmentStatus	R	ACTIVE or ENDED
startDate, endDate	LocalDate		
assignedBy, endedBy	String	R	Owner user ids
ptFeePaise	long		Optional
sessionsTotal, sessionsUsed	int		Optional PT session counter (Phase 2 usage)
notes	String		Owner remark
createdAt, updatedAt	Instant	R	

Index	Purpose
gymId + trainerId + status	Trainer visibility list
gymId + memberId + status	Member to trainer lookup

Index	Purpose
gymId + trainerId + memberId (unique, partial where status ACTIVE)	No duplicate active assignment

3.11 member_notes

Trainer notes and progress readings. Append-only with edit by author.

Field	Type	R	Notes
_id, gymId		R	
memberId	String	R	
trainerId	String	R	Author
type	NoteType	R	WORKOUT, DIET, PROGRESS, GENERAL
text	String	R	Max 2000 chars
weightKg	Double		For PROGRESS notes (health reading, not money)
measurements	Map		chest, waist, hips, arms, thighs in cm
createdAt, updatedAt	Instant	R	

Index	Purpose
gymId + memberId + createdAt desc	Timeline and chart
gymId + trainerId + createdAt desc	Trainer view

3.12 audit_logs

Who did what. Owner reads it. Never edited.

Field	Type	R	Notes
_id, gymId		R	
actorId, actorRole	String	R	
action	String	R	ASSIGN, UNASSIGN, REASSIGN, TRAINER_APPROVED, TRAINER_REJECTED, TYPE_CHANGED, PAYMENT_REVERSED, SETTING_CHANGED, MEMBER_DELETED, GYM_CODE_REGENERATED
targetType, targetId	String	R	
details	Map		Small non-PII details (old and new values for settings)
timestamp	Instant	R	

Index	Purpose
gymId + timestamp desc	Paginated view
gymId + action + timestamp desc	Filter by action

3.13 counters

Global collection with string keys. Used for atomic sequences.

Field	Type	Notes
_id	String	member:{gymId} or receipt:{gymId}:{year}
seq	long	Incremented with findAndModify \$inc, upsert true, returnNew true

4. Consistency and Denormalisation Rules

Field	Source of truth	Updated when	How
members.currentExpiry	memberships.expiryDate (latest)	Member created, renewed	Same transaction
members.currentMembershipId	Latest membership	Renewal	Same transaction
members.currentDuePaise	Sum of memberships.duePaise	Payment, reversal, renewal	Same transaction
memberships.paidPaise and duePaise	Payments of that membership	Payment, reversal	Same transaction. due = total - paid
members.lastAttendanceDate	attendance.date (max)	Check-in	After successful insert
memberships.status	expiryDate vs today	Daily job and on read	Set EXPIRED when expiryDate before today
gyms.status	subscriptionValidTill vs today and grace	Daily job and on read	TRIAL, ACTIVE, EXPIRED; SUSPENDED only by admin
payments.memberName, attendance.memberName	members.name	Not updated on rename (history keeps old name)	Intentional snapshot
trainer visibility	ACTIVE trainer_assignments	Assign or end	Read at request time, never cached on the trainer user

A nightly reconciliation job (Phase 2) recomputes denormalised fields from memberships and payments and logs mismatches. Until then, always update source and denormalised data in one transaction.

5. Query to Index Map

Query	Collection	Index used
Login by phone	users	phone
Gym by code	gyms	gymCode
Pending trainers	users	gymId + role + status
Member list with filters and pagination	members	gymId + status + currentExpiry
Search by phone or code	members	gymId + phone, gymId + memberCode
Search by name prefix	members	gymId + nameLower
Expiring in next 5 days	members	gymId + status + currentExpiry (range)
Absent for N days	members	gymId + status + lastAttendanceDate
Dues list sorted by amount	members	gymId + currentDuePaise (partial)
Today and month collection	payments	gymId + paidAt
Member payments	payments	gymId + memberId + paidAt
Duplicate payment check	payments	gymId + idempotencyKey
Today attendance and counts	attendance	gymId + date
Already checked in	attendance	gymId + memberId + date (unique)
Reminders due	memberships	gymId + status + expiryDate
Trainer visible members	trainer_assignments	gymId + trainerId + status
Member assigned trainer	trainer_assignments	gymId + memberId + status
Notes timeline	member_notes	gymId + memberId + createdAt
Audit view	audit_logs	gymId + timestamp

6. Aggregation and Query Examples

6.1 Today and month collection

```
db.payments.aggregate([
  { $match: { gymId: G, paidAt: { $gte: startOfDayUtc, $lt: startOfNextDayUtc } } },
  { $group: { _id: "$mode", totalPaise: { $sum: "$amountPaise" }, count: { $sum: 1 } } }
])
```

6.2 Last 7 days series

```
db.payments.aggregate([
  { $match: { gymId: G, paidAt: { $gte: sevenDaysAgoUtc } } },
  { $group: { _id: { $dateToString: { format: "%Y-%m-%d", date: "$paidAt",
    timezone: "Asia/Kolkata" } } },
    totalPaise: { $sum: "$amountPaise" } } },
  { $sort: { _id: 1 } }
]) // fill missing days with 0 in code
```

6.3 Absent members (trainer variant adds the assignment filter)

```
db.members.find({
  gymId: G, status: "ACTIVE",
  $or: [ { lastAttendanceDate: null }, { lastAttendanceDate: { $lt: cutoffDate } } ],
  _id: { $in: visibleMemberIds } // only for TRAINER
}).sort({ lastAttendanceDate: 1 }).limit(50)
```

6.4 Expiring soon

```
db.members.find({ gymId: G, status: "ACTIVE",
  currentExpiry: { $gte: today, $lte: today.plusDays(5) } })
.sort({ currentExpiry: 1 })
```

6.5 Any \$lookup must be tenant scoped

```
{ $lookup: { from: "members",
  let: { mid: "$memberId", g: "$gymId" },
  pipeline: [ { $match: { $expr: { $and: [ { $eq: ["$_id", "$$mid"] },
    { $eq: ["$gymId", "$$g"] } ] } } } ],
  as: "member" } }
```

7. PII Classification and Masking

Data	Class	OWNER	STAFF	TRAINER	Logs
Member name, photo	Personal	Full	Full	Full (assigned)	Id only
Member phone	Sensitive personal	Full	Masked (full if setting on)	Masked	Never
Member address, dob	Sensitive personal	Full	Omitted	Omitted	Never
Payments, dues	Financial	Full	Create, limited view	None	Ids and amounts only
Notes, weight, measurements	Health related	Read	None	Assigned only	Never
User password hash, tokens	Secret	Never	Never	Never	Never

- Masking rule: keep the last 4 digits, replace the rest with X (XXXXXX3210). Implemented in one MaskingUtils class used by role-specific DTO mappers.
- Search by full phone is allowed for STAFF (to find a person at the desk), but the response is masked.
- Exports are Owner only. Trainer export is disabled.

- Provide a way to delete or anonymise a member on request (Phase 2) while keeping payment totals.

8. Validation Constraints

Field	Rule
phone	Regex <code>^[6-9]\d{9}\$</code> after removing +91, spaces and dashes
password	Minimum 8 characters, at least one digit, maximum 72 (BCrypt limit)
name, gym name	2 to 80 characters, trimmed
gymCode	6 to 8 characters, uppercase A to Z (without I, L, O) and 2 to 9
amountPaise	Greater than 0 for PAYMENT, not more than due
durationDays	1 to 3660
pricePaise	0 to 100000000
photo	jpeg, png or webp, max 5 MB before compression, stored under 500 KB
note text	1 to 2000 characters
weightKg	20 to 400
dates	startDate not more than 30 days in the past without owner rights; expiry always after start
reason (reversal)	5 to 300 characters

9. Java Entity Skeletons

```

@Getter @Setter @SuperBuilder @NoArgsConstructor
public abstract class TenantDocument {
    @Id private String id;
    @Indexed private String gymId;
    @CreatedDate private Instant createdAt;
    @LastModifiedDate private Instant updatedAt;
}

@Document("members")
@CompoundIndexes({
    @CompoundIndex(def = "{ 'gymId':1, 'phone':1 }", unique = true),
    @CompoundIndex(def = "{ 'gymId':1, 'memberCode':1 }", unique = true),
    @CompoundIndex(def = "{ 'gymId':1, 'status':1, 'currentExpiry':1 }"),
    @CompoundIndex(def = "{ 'gymId':1, 'status':1, 'lastAttendanceDate':1 }"),
    @CompoundIndex(def = "{ 'gymId':1, 'nameLower':1 }")
})
@Getter @Setter @SuperBuilder @NoArgsConstructor
public class Member extends TenantDocument {
    private String memberCode, name, nameLower, phone, gender, address, photoPath;
    private LocalDate dob, joiningDate, currentExpiry, lastAttendanceDate;
    private MemberStatus status = MemberStatus.ACTIVE;
    private String currentMembershipId;
    private long currentDuePaise;
}

public interface MemberRepository extends MongoRepository<Member, String> {
    Optional<Member> findByIdAndGymId(String id, String gymId); // always this
    Page<Member> findByGymIdAndStatus(String gymId, MemberStatus s, Pageable p);
    boolean existsByGymIdAndPhone(String gymId, String phone);
}

public long next(String key) { // atomic counter
    return mongoTemplate.findAndModify(
        Query.query(Criteria.where("_id").is(key)), new Update().inc("seq", 1),
        FindAndModifyOptions.options().returnNew(true).upsert(true),
        Counter.class).getSeq();
}

```

Lombok: @SuperBuilder must be on the base class and on every subclass, together with @NoArgsConstructor, otherwise builders cannot set gymId and Spring Data cannot instantiate the entities. Do not mix plain @Builder with inheritance.

10. Sample Documents

```
// members
{ "_id": "66f1...a1", "gymId": "66f0...01", "memberCode": "GYM-0045",
  "name": "Rahul Sharma", "nameLower": "rahul sharma", "phone": "9876543210",
  "gender": "MALE", "joiningDate": "2026-10-07", "status": "ACTIVE",
  "currentMembershipId": "66f1...b2", "currentExpiry": "2026-11-06",
  "currentDuePaise": 50000, "lastAttendanceDate": "2026-10-07" }

// memberships
{ "_id": "66f1...b2", "gymId": "66f0...01", "memberId": "66f1...a1",
  "planId": "66f0...p1", "planName": "Monthly",
  "startDate": "2026-10-07", "expiryDate": "2026-11-06",
  "totalPaise": 100000, "paidPaise": 50000, "duePaise": 50000, "status": "ACTIVE" }

// payments (a reversal would have amountPaise -50000, type REVERSAL, reversalOfId, reason)
{ "_id": "66f1...c3", "gymId": "66f0...01", "memberId": "66f1...a1",
  "membershipId": "66f1...b2", "memberName": "Rahul Sharma",
  "amountPaise": 50000, "mode": "UPI", "type": "PAYMENT",
  "receiptNo": "GYMBRO-2026-000012", "idempotencyKey": "3f2a...e1",
  "paidAt": "2026-10-07T09:10:00Z", "receivedBy": "66f0...u1" }

// trainer_assignments
{ "_id": "66f1...d4", "gymId": "66f0...01", "trainerId": "66f0...u7",
  "memberId": "66f1...a1", "status": "ACTIVE", "startDate": "2026-10-08",
  "assignedBy": "66f0...u1", "ptFeePaise": 300000, "sessionsTotal": 12 }

// users (trainer waiting for approval)
{ "_id": "66f0...u9", "gymId": "66f0...01", "name": "Ravi", "phone": "9000011111",
  "passwordHash": "$2a$10$...", "role": "TRAINER", "status": "PENDING_APPROVAL",
  "trainerTypeIds": [], "failedAttempts": 0 }
```

11. Database Setup and Migration

11.1 Replica set for transactions

```
// mongo-init.js (runs once on first start)
rs.initiate({ _id: "rs0", members: [ { _id: 0, host: "mongo:27017" } ] });
// Start mongod with --replSet rs0 --bind_ip_all and a keyfile when auth is enabled
```

11.2 Index creation

Indexes are declared on the entities and created by Spring Data at startup (spring.data.mongodb.auto-index-creation=true). For production, create indexes with an explicit migration script and keep auto-index-creation on only in development, so that a bad deploy cannot rebuild large indexes unexpectedly.

11.3 Startup migrations (idempotent)

Migration	Action
M1 gym codes	Generate a gymCode for any gym without one
M2 user status	Convert old boolean active to status ACTIVE or INACTIVE
M3 trainer role	Existing trainers created as STAFF should be reviewed and switched to TRAINER by the owner
M4 trainer types	Seed default trainer types for existing gyms
M5 nameLower	Fill nameLower on members
M6 denormalised fields	Recompute currentExpiry, currentDuePaise, lastAttendanceDate
M7 remove city	Drop the unused city field from gyms

11.4 Retention and backups

- No automatic deletion of members, memberships or payments. INACTIVE members stay for financial history.
- Attendance older than 2 years can be archived (Phase 2). Audit logs kept 2 years.
- Daily mongodump (gzip archive), 14 days local plus off-site copy; also back up the uploads folder. Restore tested monthly.
- Estimated size: 200 members per gym with daily attendance and payments is a few MB per gym per year, so 100 gyms fit comfortably on the current server.